INSTITUTO POLITÉCTICO NACIONAL

UNIDAD PROFESIONAL INTERDISCIPLINARIA EN INGENIERÍA Y TECNOLOGÍAS AVANZADAS

“Práctica 1: Repaso de consultas sql y uso de servidores vinculados”

Peralta Ordoñez Jesús

Valencia Cedeño Marcos Gael

Asignatura: Bases de Datos Distribuidas

Profesor: De la Cruz Sosa Carlos

Grupo: 3TM2

Fecha de entrega: 23/02/2026

Desarrollo

Esta práctica utiliza la base de datos AdventureWorks de SQL Server para aplicar consultas multi-tablas, subconsultas, GROUP BY y conceptos avanzados. Los ejercicios requieren análisis de múltiples tablas como Sales.SalesOrderHeader, Sales.SalesOrderDetail, Production.Product, HumanResources.Employee y Person.Person.

Requisitos:

- Instala AdventureWorks desde el sitio oficial de Microsoft (versión OLTP).
- Usa SQL Server Management Studio (SSMS) conectado a la base de datos.
- Para cada consulta añade en el reporte el script de la consulta con discusión de la solución y captura de las 10 primeras filas del resultado.
- Subir al repositorio de cada alumno, reporte escrito de la práctica en el archivo practica1_equipoX.pdf y practica1_equipoX.SQL con scripts de las consultas.

Solución de Consultas

Consulta 1

Encuentra los 10 productos más vendidos en 2014, mostrando nombre del producto, cantidad total vendida y nombre del cliente.

1. Una vez resuelta la consulta: agrega el precio unitario promedio (AVG(UnitPrice)) y filtra solo productos con ListPrice > 1000.

```sql
USE AdventureWorks2022
GO

SELECT
    p.Name,
    T.Cant,
    per.FirstName + ' ' + per.LastName AS NombreCliente
FROM Production.Product p
JOIN (
        SELECT TOP 10 sod.ProductID,
               SUM(sod.OrderQty) AS Cant
        FROM Sales.SalesOrderDetail sod
        JOIN Sales.SalesOrderHeader soh
            ON sod.SalesOrderID = soh.SalesOrderID
        WHERE YEAR(soh.OrderDate) = 2014
        GROUP BY sod.ProductID
        ORDER BY Cant DESC
     ) AS T
    ON p.ProductID = T.ProductID
JOIN Sales.SalesOrderDetail sod2
    ON T.ProductID = sod2.ProductID
JOIN Sales.SalesOrderHeader soh2
    ON sod2.SalesOrderID = soh2.SalesOrderID
JOIN Sales.Customer c
    ON soh2.CustomerID = c.CustomerID
JOIN Person.Person per
```

```sql
     ) AS T
    ON p.ProductID = T.ProductID
JOIN Sales.SalesOrderDetail sod2
    ON T.ProductID = sod2.ProductID
JOIN Sales.SalesOrderHeader soh2
    ON sod2.SalesOrderID = soh2.SalesOrderID
JOIN Sales.Customer c
    ON soh2.CustomerID = c.CustomerID
JOIN Person.Person per
    ON c.PersonID = per.BusinessEntityID
WHERE YEAR(soh2.OrderDate) = 2014
ORDER BY T.Cant DESC;
```

Para la primera parte de la consulta se optó por el uso de una subconsulta, en la cual se seleccionan los primeros 10 productos más vendidos. Para ello, se trabaja inicialmente con la tabla Sales.SalesOrderDetail, de donde se obtienen las columnas ProductID y OrderQty.

Posteriormente, se aplica la función de agregación SUM(OrderQty) con el objetivo de calcular la cantidad total vendida por cada producto.

Una vez establecida la relación entre ambas tablas, se filtran los registros para que únicamente se consideren aquellos cuya fecha de venta pertenezca al año 2014. Después, se agrupan los resultados por ProductID. Finalmente, se ordenan los resultados de forma descendente según la cantidad total vendida y se utiliza TOP 10 para obtener únicamente los diez productos con mayor volumen de ventas durante ese año.

El resultado de esta subconsulta genera una tabla derivada que contiene el identificador del producto y la cantidad total vendida, la cual posteriormente se utiliza para realizar las uniones necesarias con otras tablas y así obtener información adicional como el nombre del producto y el nombre del cliente

Resultado

	Name	Cant	NombreCliente
1	Water Bottle - 30 oz.	2902	Isabella Adams
2	Water Bottle - 30 oz.	2902	Julia Adams
3	Water Bottle - 30 oz.	2902	Kaitlyn Adams
4	Water Bottle - 30 oz.	2902	Kaitlyn Adams
5	Water Bottle - 30 oz.	2902	Morgan Adams
6	Water Bottle - 30 oz.	2902	Anna Alexander
7	Water Bottle - 30 oz.	2902	Ian Alexander
8	Water Bottle - 30 oz.	2902	Jasmine Alexander
9	Water Bottle - 30 oz.	2902	Marcus Alexander
10	Water Bottle - 30 oz.	2902	Megan Alexander

Consulta 1.1

Una vez resuelta la consulta: agrega el precio unitario promedio (AVG(UnitPrice)) y filtra solo productos con ListPrice > 1000

```sql
SELECT
    p.Name,
    T.Cant,
    per.FirstName + ' ' + per.LastName AS NombreCliente
FROM Production.Product p
JOIN (
        SELECT TOP 10
               sod.ProductID,
               SUM(sod.OrderQty) AS Cant,
               AVG(sod.UnitPrice) AS Promedio
        FROM Sales.SalesOrderDetail sod
        JOIN Sales.SalesOrderHeader soh
            ON sod.SalesOrderID = soh.SalesOrderID
        WHERE YEAR(soh.OrderDate) = 2014
        GROUP BY sod.ProductID
        ORDER BY Cant DESC
     ) AS T
    ON p.ProductID = T.ProductID
JOIN Sales.SalesOrderDetail sod2
    ON T.ProductID = sod2.ProductID
JOIN Sales.SalesOrderHeader soh2
    ON sod2.SalesOrderID = soh2.SalesOrderID
JOIN Sales.Customer c
    ON soh2.CustomerID = c.CustomerID
JOIN Person.Person per
    ON c.PersonID = per.BusinessEntityID

WHERE YEAR(soh2.OrderDate) = 2014
  AND p.ListPrice > 1000
ORDER BY T.Cant DESC;
```

En la segunda parte de la consulta se agregó el cálculo del precio unitario promedio utilizando la función AVG(UnitPrice) dentro de la misma subconsulta donde ya se estaba calculando la cantidad total vendida. Esto se hizo porque el promedio también depende de las ventas realizadas en 2014 y del agrupamiento por ProductID, por lo que tiene sentido calcularlo en el mismo lugar donde se hace la suma.

Después, en la consulta principal, se agregó el filtro p.ListPrice > 1000. Este filtro se aplica sobre la tabla Production.Product, ya que ahí es donde se encuentra la columna ListPrice. Con esta condición solo se muestran los productos cuyo precio de lista sea mayor a 1000.

En resumen, primero se obtienen los 10 productos más vendidos en 2014 con su cantidad total y su precio unitario promedio, y luego se filtran para mostrar únicamente aquellos productos que tienen un precio de lista mayor a 1000.

Resultado

	Name	Cant	NombreCliente

Consulta 2

Lista los empleados que han vendido más que el promedio de ventas por empleado en el territorio 'Northwest'.

```sql
SELECT
Per.FirstName + ' ' + Per.LastName as nombre,
SUMA.SalesPersonID
FROM Person.Person AS Per
join(
    select SUM(ORDEN.TotalDue) AS CANT, ORDEN.SalesPersonID
    from Sales.SalesOrderHeader as ORDEN
    where ORDEN.TerritoryID = 1
    group by ORDEN.SalesPersonID
) as SUMA
on SUMA.SalesPersonID = Per.BusinessEntityID
WHERE SUMA.CANT > (
    SELECT AVG(CANT) as promedio
    FROM (
        select SUM(ORDEN.TotalDue) AS CANT, ORDEN.SalesPersonID
        from Sales.SalesOrderHeader as ORDEN
        where ORDEN.TerritoryID = 1
        group by ORDEN.SalesPersonID
    ) AS P
);
```

Primero, se utiliza una subconsulta llamada SUMA, en la cual se trabaja con la tabla Sales.SalesOrderHeader. En esta parte se agrupan los registros por SalesPersonID y se calcula la suma de TotalDue, lo que permite saber cuánto vendió cada empleado en ese territorio.

Después, esa subconsulta se une con la tabla Person.Person usando el BusinessEntityID, ya que ese campo corresponde al identificador del empleado. Esto permite obtener el nombre completo del vendedor.

Luego, en la cláusula WHERE, se utiliza otra subconsulta para calcular el promedio de ventas por empleado en el mismo territorio. Esta segunda subconsulta vuelve a agruparse por SalesPersonID y calcula la suma de ventas, y sobre esos resultados se aplica AVG(CANT) para obtener el promedio general.

Finalmente, se filtran los resultados para mostrar únicamente aquellos empleados cuya venta total sea mayor que ese promedio calculado.

Resultado

	nombre	SalesPersonID
1	Tete Mensa-Annan	284
2	Pamela Ansman-Wolfe	280
3	David Campbell	283

Consulta 2.1

Una vez resuelta la consulta convierte la subconsulta en un CTE (Common Table Expresión)

```sql
WITH VentasPorEmpleado AS
(
    SELECT
        ORDEN.SalesPersonID,
        SUM(ORDEN.TotalDue) AS Cant
    FROM Sales.SalesOrderHeader AS ORDEN
    WHERE ORDEN.TerritoryID = 1
    GROUP BY ORDEN.SalesPersonID
)

SELECT
    Per.FirstName + ' ' + Per.LastName AS Nombre,
    V.SalesPersonID
FROM VentasPorEmpleado V
JOIN Person.Person Per
    ON V.SalesPersonID = Per.BusinessEntityID
WHERE V.Cant > (
    SELECT AVG(Cant)
    FROM VentasPorEmpleado
);
```

En esta versión de la consulta se reemplazó la subconsulta repetida por una CTE (Common Table Expression), con el objetivo de hacer el código más claro y evitar repetir la misma lógica dos veces.

Primero, se define la CTE llamada VentasPorEmpleado. En ella se trabaja con la tabla Sales.SalesOrderHeader, donde se agrupan los registros por SalesPersonID y se calcula la suma de TotalDue. Esto permite obtener cuánto vendió cada empleado dentro del territorio con TerritoryID = 1.

La CTE funciona como una tabla temporal que solo existe durante la ejecución de la consulta.

Después de definir la CTE, se realiza la consulta principal. En esta parte se une VentasPorEmpleado con la tabla Person.Person mediante el campo

BusinessEntityID, ya que ese identificador corresponde al empleado. Esto permite mostrar el nombre completo del vendedor.

Finalmente, en la cláusula WHERE, se calcula el promedio de ventas usando la misma CTE y se filtran únicamente aquellos empleados cuya venta total sea mayor que ese promedio.

Resultado

	Nombre	SalesPersonID
1	Tete Mensa-Annan	284
2	Pamela Ansman-...	280
3	David Campbell	283

Consulta 3

Calcula las ventas totales por territorio y año, mostrando solo aquellos con mas de 5 ordenes y ventas>$1,000,000, ordenando por ventas descendente

```
select top (5)
soh.SalesOrderID,
soh.OrderDate,
soh.TotalDue,
st.Name as territorio
from Sales.SalesOrderHeader as soh
join Sales.SalesTerritory as st on
st.TerritoryID = soh.TerritoryID;
```

Lo que hacemos acá es unir la tabla SalesOrderHeader con SalesTerritory y relaciona cada orden con su territorio usando TerritoryID pero acá solamente devuelve 5 registros cuales quiera ya que no están agrupados

```
select
st.Name as Territorio,
year(soh.OrderDate) as Anio,
count(distinct soh.SalesOrderID) as Ordenes,
sum(soh.TotalDue) as VentasTotales
from Sales.SalesOrderHeader as soh
join Sales.SalesTerritory as st
on st.TerritoryID = soh.TerritoryID
group by
st.Name,
year(soh.OrderDate);
```

Acá realizamos un join este lo que hace es unir las tablas SalesOrderHeader y SalesTerritory para relacionar cada orden con su territorio después realizamos un group by para agrupar los registros por territorio y año esto permite calcular el numero de ordenes y el total de ventas en cada grupo

```sql
select
st.Name as Territorio,
year(soh.OrderDate) as Anio,
count(distinct soh.SalesOrderID) as Ordenes,
sum(soh.TotalDue) as VentasTotales
ambio: no guardado esOrderHeader as soh
join Sales.SalesTerritory as st
on st.TerritoryID = soh.TerritoryID
group by
st.Name,
year(soh.OrderDate)
having
count(distinct soh.SalesOrderID) > 5
and sum(soh.TotalDue) > 1000000
order by
VentasTotales desc;
```

La consulta agrupa las órdenes por Territorio y Año, calcula el número de órdenes y las ventas totales por grupo, filtra solo los grupos con más de 5 órdenes y ventas superiores a 1,000,000 usando HAVING, y ordena el resultado por VentasTotales en orden descendente.

RESULTADO

Resultados | Mensajes

	Territorio	Anio	Ordenes	VentasTotales
1	Southwest	2013	2725	10239209,3403
2	Southwest	2012	777	9329154,3425
3	Canada	2013	1884	7010449,6994
4	Northwest	2013	2053	6759500,6713
5	Canada	2012	460	6599971,0217
6	Northwest	2012	510	5325813,0562

El resultado final son 33 registros mostrando territorio, año, ordenes y ventas totales

Consulta 3.1

En el punto 1 para agregar desviación estándar, como estamos aplicando una suma de ventas por territorio, año la desviacion debe de calcularse sobre las ventas de cada orden (soh.TotalDue)

```
/*CONSULTA 3 PUNTO 1 DESVIACION ESTANDAR DE VENTAS*/
select
st.Name as Territorio,
year(soh.OrderDate) as Anio,
count(distinct soh.SalesOrderID) as Ordenes,
sum(soh.TotalDue) as VentasTotales,
stdev(soh.TotalDue) as DesvStdVentas /*aqui el uso de stdev me da cuant
from Sales.SalesOrderHeader as soh
join Sales.SalesTerritory as st
on st.TerritoryID = soh.TerritoryID
group by
st.Name,
year(soh.OrderDate)
having
count(distinct soh.SalesOrderID) > 5
and sum(soh.TotalDue) > 1000000
order by
VentasTotales desc;
```

RESULTADO

	Territorio	Anio	Ordenes	VentasTotales	DesvStdVentas
1	Southwest	2013	2725	10239209,3403	13470,4188703684
2	Southwest	2012	777	9329154,3425	23693,0432287992
3	Canada	2013	1884	7010449,6994	13359,6078579698
4	Northwest	2013	2053	6759500,6713	12654,1872740093
5	Canada	2012	460	6599971,0217	24167,4810564263
6	Northwest	2012	510	5325813,0562	20150,9169044465
7	Australia	2013	3015	4702404,0504	3719,04527457183
8	Southwest	2014	2383	4437517,8076	7343,9044753176
9	France	2013	1273	4271019,2663	12923,6957600484

Al final el resultado de la consulta fueron 33 registros MOSTRANDO la desviación estándar de ventas obtenidas en cada territorio

Consulta 4

Encuentre vendedores que han vendido todos los productos de la categoría Bikes

```sql
;with ProductosCategoria as (
    select p.ProductID
    from Production.Product as p
    join Production.ProductSubcategory as ps
       on ps.ProductSubcategoryID = p.ProductSubcategoryID
    join Production.ProductCategory as pc
        on pc.ProductCategoryID = ps.ProductCategoryID
    where pc.Name = 'Bikes'
),
ProductosVendidosPorVendedor as (
    select distinct
        soh.SalesPersonID,
        sod.ProductID
    from Sales.SalesOrderHeader as soh
    join Sales.SalesOrderDetail as sod
        on sod.SalesOrderID = soh.SalesOrderID
    where soh.SalesPersonID is not null
)
select
    sp.BusinessEntityID as SalesPersonID,
    pp.FirstName,
    pp.LastName
from Sales.SalesPerson as sp
join Person.Person as pp
    on pp.BusinessEntityID = sp.BusinessEntityID
where not exists (
    -- “No existe” un producto de Bikes que este vendedor NO haya vendido
    SELECT 1
    FROM ProductosCategoria AS pc
    WHERE not exists (
        SELECT 1
        FROM ProductosVendidosPorVendedor AS pv
        WHERE pv.SalesPersonID = sp.BusinessEntityID
          and pv.ProductID = pc.ProductID
    )
);
```

Lo que utilizamos acá es la sentencia with as el cual en la primera parte creamos ProductosCategoria el cual va contener todos los productosID pertenecientes a la categoría bikes, desde product empieza por que ahí están los productos y su productoID pero este no contiene directamente a ProductCategoryID, realiazamos un join entre ProductSubCategory y productCategoryID, después filtramos para quedarme solo con productos cuya categoría sea bikes

Para el de abajo crea pares que indican los productos que ha vendido cada vendedor

RESUTADO

Resultados | Mensajes

	SalesPersonID	FirstName	LastName
1	279	Tsvi	Reiter
2	277	Jillian	Carson
3	276	Linda	Mitchell
4	282	José	Saraiva
5	281	Shu	Ito

Consulta ejecutada correctamente

Consulta 4.1

aquí cambiamos la categoría “Clothing” (ID=4). Lo único que cambiamos acá es esta línea de código WHERE pc.ProductCategoryID = 4

RESULTADO

Al salir la consulta como vacia significa que ni un vendedor vendio todos los productos, al yo cambiarle el clothing mi consulta es “Dame los vendedores que hayan vendido todos los productos de la categoría bikes” eso significa que si un vendedor no vendio todos los productos al final en la consulta aparecerá fuera de esta ya que no cumple con lo indicado

Consulta 4.2

En esta consulta ya no se buscan todos los registros y ahora queremos saber cuantos distintos productos han vendido cada vendedor en cada categoría en esta consulta relacionamos las siguientes tablas con joins

SalesPerson, SalesOrderHeader, SalesOrderDetail, Product, ProductSubcategory, ProductCategory

Utilizamos un count distinct para contar productos diferentes (vendedor,categoría)

```sql
select
    sp.businessentityid as salespersonid,
    pp.firstname,
    pp.lastname,
    pc.name as categoria,
    count(distinct sod.productid) as productosdistintosvendidos
from sales.salesperson as sp
join person.person as pp
    on pp.businessentityid = sp.businessentityid
join sales.salesorderheader as soh
    on soh.salespersonid = sp.businessentityid
join sales.salesorderdetail as sod
    on sod.salesorderid = soh.salesorderid
join production.product as p
    on p.productid = sod.productid
join production.productsubcategory as ps
    on ps.productsubcategoryid = p.productsubcategoryid
join production.productcategory as pc
    on pc.productcategoryid = ps.productcategoryid
group by
    sp.businessentityid,
    pp.firstname,
    pp.lastname,
    pc.name
order by
    sp.businessentityid,
    pc.name;
```

RESULTADO

	salespersonid	firstname	lastname	categoria	productosdistintosvendidos
16	277	Jillian	Carson	Components	107
17	278	Garrett	Vargas	Accessories	10
18	278	Garrett	Vargas	Bikes	96
19	278	Garrett	Vargas	Clothing	32
20	278	Garrett	Vargas	Components	96
21	279	Tsvi	Reiter	Accessories	10

La salida de esta consulta indica el numero de productos distintos que ha vendido cada vendedor en cada categoría

Consulta 5

Determinar el producto más vendido de cada categoría de producto, considerando el escenario de que el esquema SALES se encuentra en una instancia (servidor) A y el esquema PRODUCTION en otra instancia (servidor) B.

```sql
USE AdventureWorks2022
GO
SELECT name, cant
FROM Production.Product p
JOIN (
        SELECT TOP 10 productid, SUM(orderqty) cant
        FROM SV_GAEL.AdventureWorks2022.Sales.SalesOrderDetail sod
        GROUP BY productid
        ORDER BY cant DESC
     ) AS T
ON p.ProductID = T.ProductID

SELECT soh.SalesOrderID, sod.ProductID, sod.OrderQty, soh.CustomerID
FROM SV_GAEL.AdventureWorks2022.Sales.SalesOrderHeader soh
JOIN SV_GAEL.AdventureWorks2022.Sales.SalesOrderDetail sod
ON soh.SalesOrderID = sod.SalesOrderID
WHERE YEAR(OrderDate) = '2014'
```

```sql
EXEC sp_addlinkedserver
   @server = 'SV_GAEL', /*Nombre del servidor a asociar*/
   @srvproduct = 'SQLServer', -- opcional
   @provider = 'MSOLEDBSQL19',
   @datasrc = '172.20.10.4',
   @provstr = 'Encrypt=yes;TrustServerCertificate=yes;';

EXEC sp_addlinkedsrvlogin
   @rmtsrvname = 'SV_GAEL',
   @useself = 'false', -- valor false si se usarán credenciales distintas
   @rmtuser = 'sa', /*Nombre del servidor a asociar*/
   @rmtpassword = '1234';

EXEC sp_testlinkedserver SV_GAEL;
```

100 % No se encontraron problemas.

Resultados Mensajes

	name	cant
1	AWC Logo Cap	8311
2	Water Bottle - 30 oz.	6815
3	Sport-100 Helmet, Blue	6743
4	Long-Sleeve Logo Jersey, L	6592
5	Sport-100 Helmet, Black	6532
6	Sport-100 Helmet, Red	6266
7	Classic Vest, S	4247
8	Patch Kit/8 Patches	3865
9	Short-Sleeve Classic Jersey, XL	3864
10	Long-Sleeve Logo Jersey, M	3636

	SalesOrderID	ProductID	OrderQty	CustomerID
1	63363	932	1	14685
2	63363	873	1	14685
3	63364	878	1	18299
4	63364	877	1	18299
5	63365	921	1	21279
6	63366	881	1	14518
7	63366	712	1	14518
8	63367	796	1	13605
9	63367	707	1	13605
10	63368	983	1	14170
11	63368	867	1	14170
12	63368	875	1	14170
13	63369	782	1	17182

Para el desarrollo de la consulta 5 se tuve que hacer uso de una subconsulta, en la cual con la función de agregación SUM y con Select TOP 10, se encontraron los 10 productos mas vendidos de la empresa, esto sucede porque sum este encargado de contar la cantidad de filas que hay en Orderqty y haciendo suma de su contenido, posteriormente con GROUP BY agrupamos todo por Productoid, es decir, las filas

repetidas son juntadas en una sola y por último los ordenamos de forma descendente. Al terminar la subconsulta obtendremos una tabla derivad a la cua; nombramos con el alias de T y unimos ambas tablas, la tabla temporal T con la tabla original en donde coincida productoid